Progression, Loot, Skein Weaving, and Character Creation

Status

This document is the canonical design direction for character progression, build customization, loot generation, and future character creation.

The principles and system relationships below are confirmed. Exact level caps, curves, slot counts, drop rates, stat budgets, reward values, and other numeric tuning remain **TBD** until combat prototypes and multiplayer playtests produce evidence. Any number shown in an example is illustrative, not canon.

Design Goals

- Make advancement permanent, earned, and clearly felt in combat.
- Reward difficult PvE and dangerous faction conflict in proportion to their challenge.
- Let equipment improve every part of battle without allowing equipment to replace aim, timing, positioning, blocking, dodging, interrupts, or judgment.
- Give each class multiple viable directions without copying a traditional point-and-row talent tree.
- Give each faction distinct skills and utilities that affect character development without making one faction universally stronger.
- Keep every persistent reward server-owned, reproducible, auditable, and safe to retry.
- Prepare the data model for full character creation even while the prototype uses a fixed character.

The game should not manufacture engagement through a constant stream of trivial rewards. Upgrades may be less frequent, but a meaningful reward should produce a noticeable increase in power, speed, resilience, efficiency, or tactical choice.

Shared Combat-State Contract

Progression, equipment, and Skein rules consume the shared combat grammar; they do not redefine it. The stable resources are Health (`combat.resource.health`), Endurance (`combat.resource.endurance`), Guard (`combat.resource.guard`), Ward (`combat.resource.ward`), and the selected OrderResource (`combat.resource.order`). Wrought (`combat.damage.wrought`), Cinder (`combat.damage.cinder`), and Wake (`combat.damage.wake`) are the stable damage families.

- Health determines life state. An item or progression rule may modify an authored Health bound or restoration path. Any future deliberate Health cost or death-prevention rule must be explicit and readable rather than inferred from another resource.
- Endurance is shared exertion, Guard is active defensive stability, Ward is explicit absorb protection, and OrderResource is the selected Order's distinct currency. A modifier names exactly which resource it affects; broad labels such as stamina, mana, shield, or class resource are not interchangeable data keys.
- Damage modifiers name a family or explicitly name all families. No generic display phrase silently changes Wrought, Cinder, and Wake together, and a family name alone grants no armor bypass, control, damage-over-time, or other secondary behavior.
- Resource restoration is passive, delayed, event-driven, or effect-supplied as authored. Action recovery and cooldowns remain separate. Every rate, delay, threshold, cap, cost, and reset value is tuning `TBD` until its owning implementation and evidence work approves it.

Every duration effect used by progression or equipment declares a stable effect identity, beneficial/harmful/control classification, stacking key, source scope, stack cap, duration/refresh/replace rule, cleanse eligibility, and proc eligibility. Cleanses operate on explicit server-owned categories rather than a client-selected hidden effect. Resolve is the shared server-owned protection against repeated eligible control, not generic damage reduction or universal immunity. Its gain, decay, consumption, immunity interaction, and every numeric value remain tuning `TBD`.

Triggered effects carry their authoritative source and activation ancestry. The server enforces bounded depth plus aggregate listener, event, target, and construct budgets for the entire root activation, rejects recursive self-triggering and duplicate grants, and records deterministic ordering when more than one trigger is eligible. Exact values remain tuning `TBD`. Equipment, Forms, Threads, Keystones, and Doctrines cannot create a client-authored hit, target, resource result, effect, progression change, or hidden-state disclosure.

Permanent Character Progression

Character Level

Character Level is permanent progression. It represents training and mastery and must never be reset by a faction campaign, PvP season, death, or equipment change.

Character Level may unlock:

- Base attributes and resource growth.
- Class abilities and deeper combat interactions.
- Equipment and content tiers.
- Skein Forms, Threads, and Keystones.
- Access to increasingly dangerous regions and encounters.

The maximum level, experience curve, unlock schedule, and level requirements are TBD. Levels must introduce complexity gradually rather than presenting the full combat system at character creation.

Whether Character Level is a prerequisite for any Faction Doctrine option is an open design decision. Do not encode an assumed relationship before it is resolved.

Ember Rank

Ember Rank is separate from Character Level. It expresses a character's attunement, story standing, faction achievements, and visible place in the world. It is seasonal: it may compress or reset between Ember Cycles, but it must not remove permanent Character Levels or learned core options.

Its exact ranks, season length, reset or compression rules, and rewards are TBD. Suitable rewards include titles, aura or armor evolution, access, and faction recognition in addition to carefully bounded power.

Progress Must Remain Visible

Advancement should be apparent in both performance and presentation:

- Earlier threats become easier as the character becomes stronger; universal scaling must preserve the feeling of growth.
- New Forms or Threads create additional decisions and combo routes.
- Gear can improve damage, defense, recovery, resources, support, and controlled forms of speed.
- Major milestones can change Ember effects, armor details, titles, or ability presentation.

Difficulty should come from readable mechanics and capable opponents, not hidden rules or unavoidable damage.

Skein Weaving

Skein Weaving replaces a conventional talent tree. It is a loadout of interacting combat rules, not a sequence of rows filled with small percentage bonuses.

In lore, the Skein is how an Emberbound draws on the Deepwake's unrealized histories: a Form is an alternate history of a technique, a Thread binds a remembered cause to its consequence, and a Keystone accepts one defining contradiction into the character. The lore framing never changes the server-authoritative mechanics below.

Every class first receives a complete, functional base kit. Skein choices then change how that kit behaves and how one successful action can lead into another. No class should require a specific Skein choice merely to perform its basic role. Classes are the five Orders defined in Characters and Factions; Oathscar is the active initial Order working label, superseding Skeinblade.

Ability Forms

A Form changes the behavior of one ability. A Form should alter a decision, targeting pattern, timing window, movement option, resource interaction, or team function rather than only adding a small statistic.

Illustrative examples:

- An Oathscar thrust becomes a short advancing strike or a stationary guard-break tool.
- A Hushblade dodge favors repositioning behind a target or preserves momentum for the next combo.
- A Nullwright projectile becomes a direct burst or a delayed area-control effect.
- A Gravecant ward becomes stronger on one ally or weaker across a wider area.

Only one selected Form may modify a given ability at a time unless a later system explicitly supports compatible combinations.

Threads

Threads connect combat verbs. They reward execution by making one event alter a later action.

Illustrative examples:

- Perfect Block -> the next spear strike deals greater Guard damage.
- Perfect Dodge -> the next attack gains a repositioning step.
- Successful Interrupt -> a movement ability recovers sooner.
- Complete a combo without missing -> transform its finisher.
- Save a low-health ally -> create a short-lived ward.

Threads should use clear triggers, short readable state, and visible feedback. They must not create hidden chains an opponent cannot understand.

Keystones

A Keystone changes a class's central rule and anchors a build. It should produce a recognizable playstyle rather than a passive numerical advantage.

Possible design spaces include:

- Converting a defensive resource into counterattack pressure.
- Trading immediate burst for sustained control.
- Changing how momentum is gained and spent.
- Turning direct healing into prepared wards or the reverse.

Keystone effects, acquisition, and compatibility are TBD.

Order Specializations and Peaks

Each Order carries two specializations and one Peak with stable semantic IDs, recorded in Characters and Factions. A specialization selects the dominant interpretation of the Order's oath and shapes which Forms, Threads, and Keystones suit a build; a Peak briefly realizes a forbidden version of the character.

Each Order also names one combat resource: Grudge (Oathscar), Proof (Nullwright), Tempo (Hushblade), Cadence (Gravecant), and Tension (Blackfletch). All display names are working names. Specialization mechanics, Peak mechanics, resource behavior, and every numeric value are tuning TBDs; the five names are presentations of the shared `combat.resource.order` slot, not substitutes for Health, Endurance, Guard, or Ward. Specializations and Peaks extend the Order layer and do not replace the Forms, Threads, Keystone, and Faction Doctrine structure.

Faction Doctrine

Faction Doctrine supplies faction-specific skills and utilities that influence character development. Race does not determine Doctrine; faction membership does.

Doctrine may affect:

- Mobility and traversal.
- Scouting, concealment, or detection.
- Defensive wards, banners, and formations.
- Siege attacks, fortification, and invasion support.
- Resource transport, recovery, or disruption.
- A bounded combat utility that interacts with the class Skein.

The factions should be asymmetrical in method but comparable in total opportunity. One faction must not become the universal damage choice while the other is confined to defense. Every class must remain viable in either faction.

Faction-specific Forms or Threads may be added later, but the initial design should keep Doctrine as a distinct slot so class and faction balance can be tuned independently.

Initial Scope

The first implementation should contain only enough choices to prove that weaving changes real combat decisions:

- A small set of primary class abilities with mutually exclusive Forms.
- A small equipped Thread loadout chosen from a larger unlocked collection.
- One equipped Keystone.
- One equipped Faction Doctrine when the faction stage introduces it.

- A clear loadout screen that shows every active connection.

An illustrative prototype configuration could use four primary abilities, two Forms per ability, three Thread slots, one Keystone slot, and one eventual Doctrine slot. Those counts are examples only; the final counts are TBD.

Changing an equipped weave should occur at a defined safe service, such as a city trainer or rest point, during the initial implementation. Costs, field changes, saved loadouts, and competitive restrictions are TBD.

Doctrine enters the playable delivery only with faction selection. Before the 2.0 faction stage, a persisted character has `Faction = Unassigned`, cannot equip a Doctrine, and remains valid without one.

Unlocking and Extensibility

- Core Forms, Threads, and Keystones unlock predictably through permanent progression and accomplishments; build-critical options must not depend only on a random drop.
- Faction Doctrine options unlock through faction progression and faction content.
- Equipment may strengthen or alter a weave interaction, but it must not be the sole source of the class's core identity.
- New Forms, Threads, Keystones, and Doctrines should be data-driven additions, not new branches grafted onto a fixed tree.
- Persistent state stores unlocked options separately from the currently equipped weave so future loadouts can reuse the same collection.

Combat Power and Equipment

Power Relationship

Combat outcome should be determined by three interacting layers:

1. **Character progression** defines the available foundation.
2. **Equipment and Skein choices** define advantages and tactical tools.
3. **Player execution** determines how much of that potential is realized.

Gear must matter. A stronger weapon, better armor, or well-built set should be felt immediately. It must not create automatic hits, automatic defense, or unreadable attacks. Within a supported power band, a highly skilled player in weaker gear should have a credible chance against a less skilled player in stronger gear.

Large progression gaps may still be significant. Their exact effect is a playtest and balance decision, not a fixed percentage in this document.

What Items May Improve

- Weapon damage and ability output.
- Armor, health, resistance, and Guard.
- Endurance, Guard, Ward, and the selected OrderResource.
- Attack cadence, recovery, cooldowns, and resource efficiency.
- Blocking, dodging, interruption, stagger, and Guard damage.
- Healing, shielding, support, and group utility.
- Conditional effects tied to Forms, Threads, and combat execution.
- PvE encounter utility, faction logistics, and siege utility.
- Movement through controlled effects such as sprint efficiency, conditional bursts, or recovery after a successful action.

Raw movement, attack, and recovery speed require strict budgets and caps. Excessive speed would undermine animation readability, hit validation, network prediction, and counterplay.

Power Bounds

Every item is constrained by a source tier and a total power budget:

- Content defines an allowed item-power band.
- Rarity changes how the budget is distributed; it does not provide unlimited power.
- Affixes consume budget and obey category and stacking rules.
- Signature effects reserve part of the item's budget.
- Percentage modifiers use named stacking groups and server-enforced caps.
- Effects must preserve the readable wind-up, commitment, and counter window of a combat action.
- Supported PvP power gaps, time-to-defeat bounds, and speed caps are TBD and must be established through latency-aware combat tests.

The same server-authoritative rules apply in PvE and open-world PvP. Any future mode-specific coefficient must be explicit and justified; it must not silently replace a player's equipment.

Item Model

A generated combat item is composed from authoritative data:

```
Item =
    Base Template
    + Source Tier and Item Power
    + Quality
    + Affix Budget and Affixes
    + Optional Signature Effect
    + Persistent Instance Identity
```

The names and number of quality tiers are TBD. Quality is not itself a promise that an item is useful; item power, affix fit, and signature behavior all matter.

Named bosses may have recognizable signature items with fixed identity and a controlled amount of affix variation.

Reward Sources

- Ordinary enemies primarily provide currency, materials, and occasional equipment.
- Elites provide stronger source tables and themed materials.
- Quests and progression milestones provide predictable rewards.
- Dungeon bosses provide a guaranteed baseline reward plus boss-specific rolls.
- World bosses provide personal, contribution-qualified rewards.
- PvE equipment rewards use personal loot by default, so one character's roll does not consume another character's reward.
- Contested territories improve eligible source tables because players accept mandatory open-world PvP risk.
- Faction objectives, invasions, bounties, and defense provide faction rewards.
- Unwanted items can feed a salvage or upgrade path; its exact economy is TBD.

There is no requirement to issue a reward on a timed cadence. Reward value comes from the challenge, risk, and accomplishment that produced it.

Delivery Staging

- The 1.0 cooperative vertical slice persists permanent progress and bounded integer currency rewards through transactional, idempotent receipts.
- Generated item inventory and equipment begin in 1.1 with item ownership, versioning, migration, and reconciliation.
- Deferring item rewards from 1.0 does not change the target loot design in this document.

Server-Side Drop Algorithm

Requirements

The reward service must be:

- **Authoritative:** clients never choose, roll, create, or grant an item.
- **Deterministic:** the same validated reward event and roll inputs reproduce the same result.
- **Idempotent:** retrying a completed event returns its recorded result and never grants a duplicate.
- **Tunable:** tables, weights, budgets, and caps are versioned data.
- **Auditable:** the server records why the player was eligible, which table version was used, and what was granted.
- **Transactional:** the reward, inventory mutation, history update, and bad-luck update succeed or fail together.

Authoritative Inputs

```
Reward Event ID
Character ID
Source ID and source type
Loot-table version
Encounter tier and difficulty
Contested-risk state
Server-validated contribution and mastery
Class and usable item categories
Recent reward history
Bad-luck state
Server-only random-seed key and key version
```

The random stream is derived from a server-only key and stable event data, for example through a keyed pseudorandom function. The server stores the resulting reward receipt. A client-provided seed, timestamp, claimed damage value, or claimed kill is never trusted.

Generation Flow

```
GenerateReward(event, character):
    validate event, source, completion, contribution, and eligibility

    if a reward receipt already exists for event + character:
        return the recorded receipt

    table = load the event's versioned loot table
    rng = derive server random stream from event + character + table version

    result = table.guaranteed rewards
    rewardBudget = table.base budget
    rewardBudget += validated difficulty, mastery, and contested-risk modifiers
    rewardBudget = clamp to the source's allowed budget

    for each roll defined by the table:
        categoryWeights = apply bounded smart-loot weighting
        category = weighted choice(categoryWeights, rng)

        qualityWeights = table.base quality weights
        qualityWeights = apply capped source and bad-luck adjustments
        quality = weighted choice(qualityWeights, rng)

        template = weighted choice(valid templates for category and source, rng)
        itemPower = choose within the source's item-power band
        affixBudget = calculate from itemPower, quality, and signature cost
        affixes = choose valid non-conflicting affixes within affixBudget

        add the generated item to result

    atomically:
        persist result and reward receipt
        grant inventory, currency, and progression changes
        update recent-reward and bad-luck state

    return receipt
```

Exact formulas and weights are TBD. They belong in versioned data and telemetry, not hard-coded constants.

Eligibility and Mastery

Eligibility is based on server-observed participation. Depending on the activity, valid contribution can include:

- Damage and meaningful control.
- Healing, shielding, blocking, and ally protection.
- Interrupts and completion of encounter mechanics.
- Objective capture, defense, transport, or siege work.
- Presence for required encounter phases without exploitative inactivity.

Mastery may improve a reward budget for optional mechanics or exceptional execution, but it must preserve the encounter's baseline earned reward. Mastery definitions and bonuses are TBD per encounter.

Smart Loot and Bad-Luck Fairness

These safeguards prevent pathological outcomes; they are not a promise of constant gratification.

Smart Loot

- A tunable portion of equipment rolls favors items usable by the current class and build.
- The remaining pool can contain universal, trade-eligible, crafting, or off-build rewards according to the source table.
- Smart loot never considers race or sex.
- It does not guarantee a perfect item or eliminate discovery.
- Recent exact duplicates may be down-weighted when the source design allows it.

The smart-loot proportion and duplicate rules are TBD.

Bad-Luck Handling

- Important reward families may track failures on the server.
- Repeated eligible completions can add a capped adjustment or eventually grant a source currency used to choose a reward.
- Different boss signatures or reward families may keep separate counters.
- Receiving the protected reward updates or resets its relevant state in the same transaction as the grant.
- Ineligible participation and exploit-rejected events never advance protection.

Protection establishes a fair upper bound on extreme bad luck. It does not guarantee frequent rare drops, lower encounter difficulty, or reward non-participation. Thresholds, caps, currencies, and reset rules are TBD.

Risk and Reward

Dangerous content should improve opportunity without invalidating safer progression:

- Encounter difficulty raises the available reward budget or table tier.
- Optional mechanics may add a mastery modifier.
- Contested territories add a bounded risk modifier while mandatory PvP is active.
- Major faction objectives and capital invasions have their own reward tables.
- Risk bonuses never exceed the power band assigned to the content tier.
- A losing player retains permanent levels, learned Skein options, and equipped items.

In contested territory, a defeated player may lose a bounded portion of unbanked resources gathered during that expedition. Equipped items, quest-critical objects, and permanent progression remain with the character. The resource types, banking rules, and loss fraction are TBD.

Open-World PvP Rewards

Direct player kills should not independently generate random high-power equipment. That would encourage arranged kill trading and farming of vulnerable players.

Open-world PvP rewards instead derive from:

- Server-validated faction objectives and event completion.
- Contribution to an invasion or defense.
- Assists, protection, healing, scouting, transport, and siege actions.
- Bounties on dangerous enemy players.
- A bounded transfer of eligible unbanked contested resources.
- Faction currency or reward caches tied to completed objectives.

Reward calculation must apply:

- Rapidly diminishing or zero value for repeatedly defeating the same player.
- No reward for protected respawns or invalid targets.
- Strong penalties for collusion and traded kills.
- Eligibility and contribution thresholds.
- Rules for severe level or power disparity.
- Greater weighting for objectives than for unrelated kill farming.

Exact formulas, disparity thresholds, repeat-victim windows, and reward values are TBD. Equipped gear remains with its owner after PvP death.

Future Character Creation

Every playable race supports both male and female characters. Race and sex are visual and cultural identity choices; they do not restrict class, faction, Skein direction, attributes, loot probability, or combat power.

The canonical anatomy, appearance options, rig rules, equipment fitting, and art-validation requirements are defined in Playable Peoples.

Logical Character Data

```
Character
|-- Identity
|   |-- Character ID
|   `-- Name
|-- Appearance
|   |-- Race: Aurin, Kell, or Vesh
|   |-- Sex: Male or Female
|   |-- Body Preset
|   |-- Constrained Visual-Height Preset
|   |-- Age-Presentation Preset
|   |-- Face Preset
|   |-- Skin Tone (Surface Tone for Kell; Coat Tone for Vesh)
|   |-- Markings
|   |-- Voice
|   `-- Race Features
|       |-- Aurin: Hair Style, Hair Color, and Hair-Dye Treatment
|       |-- Kell: Horn Form, Plate Pattern, Mineral Tones, Mineral Finish, Wear,
|       |   Engraving, and Inlay
|       `-- Vesh: Hair Style, Coat Texture, White-Fur Pattern, Face-Surface Finish
|-- Permanent Progression
|   |-- Character Level
|   `-- Experience
|-- Seasonal Progression
|   `-- Ember Rank and Ember Cycle
|-- Class and Skein
|   |-- Class
|   |-- Learned Abilities
|   |-- Unlocked Forms, Threads, and Keystones
|   `-- Equipped Skein Weave
|-- Faction and Doctrine
|   |-- Faction: Unassigned before 2.0, then selected faction
|   |-- Doctrine Unlocks
|   `-- Equipped Doctrine
|-- Inventory and Equipment
|-- Cosmetics
`-- Session and Location
```

The approved one-time faction selection determines the faction starting territory, story, and Doctrine. Doctrine is unavailable while faction is unassigned. Any later faction-change policy is TBD.

Male and female presentations may use different bodies, faces, hair, voices, and animations. Combat collision, reach, trace rules, timing, and readable telegraphs must remain equivalent so no presentation becomes a competitive advantage.

All people and sexes share one authoritative combat capsule, one combat-presentation skeleton, weapon-socket alignment, camera and aim-presentation anchors, a server-authored trace model, contact timing, and root-motion distance. Kell horn, ear, and plate attachments and any auxiliary visual bones, together with Vesh hair and fur presentation bones, serve presentation only. They never affect collision, authoritative traces, shared weapon-presentation sockets, reach, or gameplay state. No presentation socket establishes a hit volume, window, contact, or selected target.

One equipment item has one gameplay identity and race-and-sex-fitted visual assets. Its fitting set must support every approved body, height, hair, horn, plate, fur, and natural-face combination, including Kell external ears and clawed extremities and Vesh clawed three-digit hands and digitigrade, three-toed paws, without removing biological anatomy to fit equipment or restricting equipment access.

Portable cultural pigments and adornment styles belong to shared cosmetic, marking, or equipment layers rather than a race-restricted appearance field.

The prototype may use one fixed character, but gameplay code and persistent data must not embed that character's race, sex, appearance, or faction inside class abilities. Separating these concerns now preserves future character creation.

Data and Security Boundaries

The server owns and validates:

- Experience, levels, Ember Rank, and unlocks.
- Equipped Forms, Threads, Keystones, and Doctrine.
- Item creation, affixes, power, ownership, and equipment state.
- Encounter eligibility, mastery, contested risk, and PvP contribution.
- Reward receipts, history, bad-luck state, currency, and resource loss.

The client may request an allowed loadout change or display a predicted reward animation. It cannot grant progression, choose a drop result, alter item data, or assert that an objective or opponent was defeated.

Every persistent mutation needs a stable request or event ID, authorization, validation, an atomic transaction, and an audit record.

The persistence implementation additionally requires a single active character authority lease with a monotonically increasing fencing epoch, optimistic state versions, an immutable reward/currency ledger, and a transactional outbox. The canonical technical rules live in Progression and Persistence

Architecture.

Acceptance Criteria for Future Implementation

Progression and Skein

- Character Level survives reconnects, season changes, deaths, and equipment changes.
- The base class remains functional with no Skein choices equipped.
- Each implemented Form or Thread changes an observable combat decision.
- Invalid combinations and unavailable options are rejected by the server.
- Faction Doctrine works with every supported class and provides no universal faction advantage.

Loot

- Replaying the same reward event cannot duplicate a grant.
- The recorded event inputs and table version reproduce the generated result.
- Generated items never exceed their source power or affix budgets.
- Invalid affix combinations and stat-cap violations are rejected.
- Smart-loot and bad-luck state update only for eligible, completed events.
- PvP repeat-kill and collusion cases yield no exploitable reward.
- Gear changes performance noticeably while preserving readable counterplay.

Character Creation

- Every playable race offers male and female choices.
- Both choices can use every supported class and faction.
- Appearance never changes combat collision, reach, timing, statistics, or drop chance.
- Every saved race/sex/body combination validates against its race-specific appearance schema.
- Supported equipment appearances fit every people and sex without hiding mandatory anatomy, including extreme supported customization combinations.
- Race-specific animation presentation preserves alignment with shared presentation sockets, server-authored traces, contact frames, root motion, recovery, and readable telegraphs.

Tuning Decisions Still Open

- Character Level cap, experience curve, and unlock cadence.
- Relationship between Character Level and Doctrine prerequisites.
- Ember Rank names, rewards, season length, and reset or compression rules.
- Exact Form, Thread, Keystone, and Doctrine collections.
- Number of equipped choices and loadout-change rules.

- Item tiers, quality names, stat budgets, caps, and stacking groups.
- Loot-table weights, smart-loot weighting, and duplicate handling.
- Bad-luck thresholds, caps, source currencies, and reset behavior.
- Contested-risk modifier and unbanked-resource loss rules.
- PvP contribution, disparity, anti-collusion, and repeat-victim formulas.
- Trading, binding, crafting, salvage, durability, and upgrade economies.

These decisions require a working combat model, server-authority tests, and measured player outcomes. They must not be guessed before that evidence exists.